


Síntesis, Timing y Arquitecturas de Sumadores

Diseño e Implementación de Circuitos Digitales en VLSI

Agustín Pesce - José Torres - Santiago Garaventa — Becas de Grado 2026

Mayo 2026

Contents

1 Setup Time y Hold Time	2
1.1 El flip-flop y sus restricciones de timing	2
1.2 El timing path: de flip-flop a flip-flop	2
1.3 Slack: cuánto tiempo sobra o falta	2
1.4 Setup check vs Hold check	3
1.5 Corners: el chip en distintas condiciones	3
1.6 SDC: las constraints de timing	3
2 Tipos de celdas estándar: Multi-Vt	4
2.1 ¿Qué es el Voltage Threshold (Vt)?	4
2.2 Familia de celdas Multi-Vt	4
2.3 Descripción de cada familia	4
2.4 Resumen comparativo completo	5
2.5 Multi-Vt en SAED 32nm vs tecnologías avanzadas	5
2.6 Cómo el sintetizador usa Multi-Vt	5
3 Arquitecturas de sumadores	5
3.1 Ripple Carry Adder (RCA)	5
3.2 Carry Lookahead Adder (CLA)	6
3.3 Carry Save Adder (CSA)	6
4 Experimento: barrido de síntesis	7
4.1 Setup del experimento	7
4.2 Resultados	8
4.3 Análisis: Slack vs Frecuencia	8
4.4 Análisis: Área vs Frecuencia	9
4.5 Análisis: HVT vs LVT	9
4.6 Frecuencia máxima por ancho	10
5 Leyendo un timing report	10
6 Conclusiones	11

1 Setup Time y Hold Time

Antes de hablar de síntesis, necesitamos entender **por qué** un circuito puede fallar a frecuencias altas. La respuesta está en los tiempos de setup y hold del flip-flop.

1.1 El flip-flop y sus restricciones de timing

Un flip-flop no puede capturar datos en cualquier momento. El dato en la entrada D tiene que estar **quieto** (estable) en una ventana alrededor del flanco del clock. Si el dato cambia dentro de esa ventana, el flip-flop entra en **metaestabilidad**: la salida Q queda en un voltaje intermedio entre 0 y 1, ni uno ni cero, durante un tiempo impredecible.

Esa ventana prohibida tiene dos componentes:

- **Setup time** (T_{setup}): cuánto tiempo *antes* del flanco del clock tiene que estar estable el dato D. Si D cambia durante este período, el flip-flop no tiene tiempo de prepararse para capturar el valor.
- **Hold time** (T_{hold}): cuánto tiempo *después* del flanco del clock tiene que mantenerse estable el dato D. Si D cambia demasiado pronto después del flanco, el flip-flop puede capturar el valor nuevo en vez del viejo.

Estos tiempos son propiedades **físicas** del flip-flop, determinados por la tecnología de fabricación. En SAED 32nm, un flip-flop típico tiene $T_{\text{setup}} \approx 0.10 \text{ ns}$ y $T_{\text{hold}} \approx 0.04 \text{ ns}$.

1.2 El timing path: de flip-flop a flip-flop

Todo diseño digital sincrónico se puede pensar como flip-flops conectados por lógica combinatorial. El análisis de timing se hace sobre **paths** entre pares de flip-flops:

El dato sale de FF1 (con un delay $T_{\text{clk} \rightarrow \text{Q}}$), atraviesa la lógica combinatorial (con delay T_{comb}), y tiene que llegar a FF2 con suficiente anticipación (T_{setup}) antes del próximo flanco de clock.

Si la suma de todos los delays supera el período del clock, el dato llega tarde y FF2 captura basura.

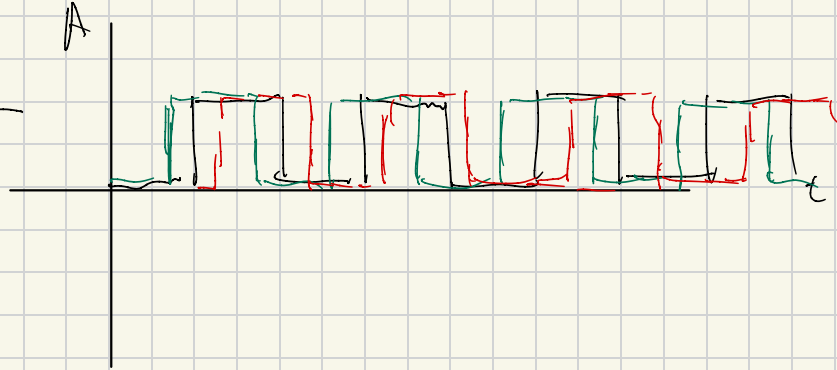
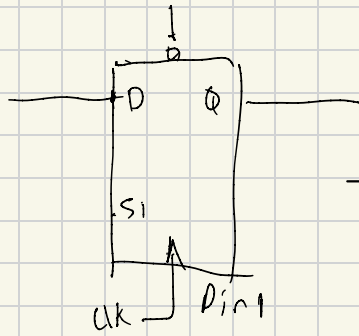
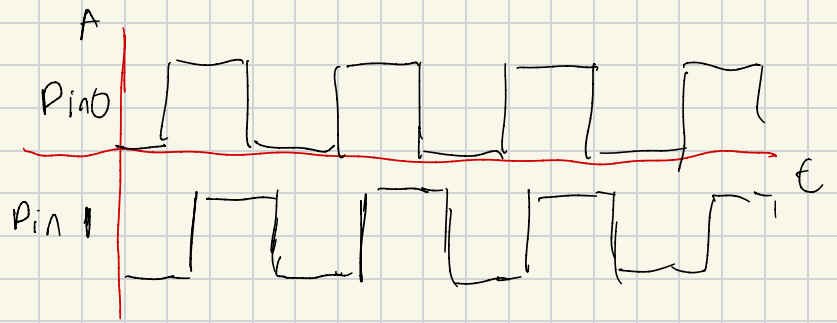
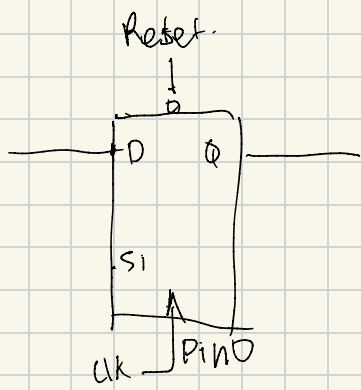
1.3 Slack: cuánto tiempo sobra o falta

El **slack** es la diferencia entre cuándo *tiene* que llegar el dato y cuándo *realmente* llega:

$$\boxed{\text{Slack} = T_{\text{data required}} - T_{\text{data arrival}}}$$

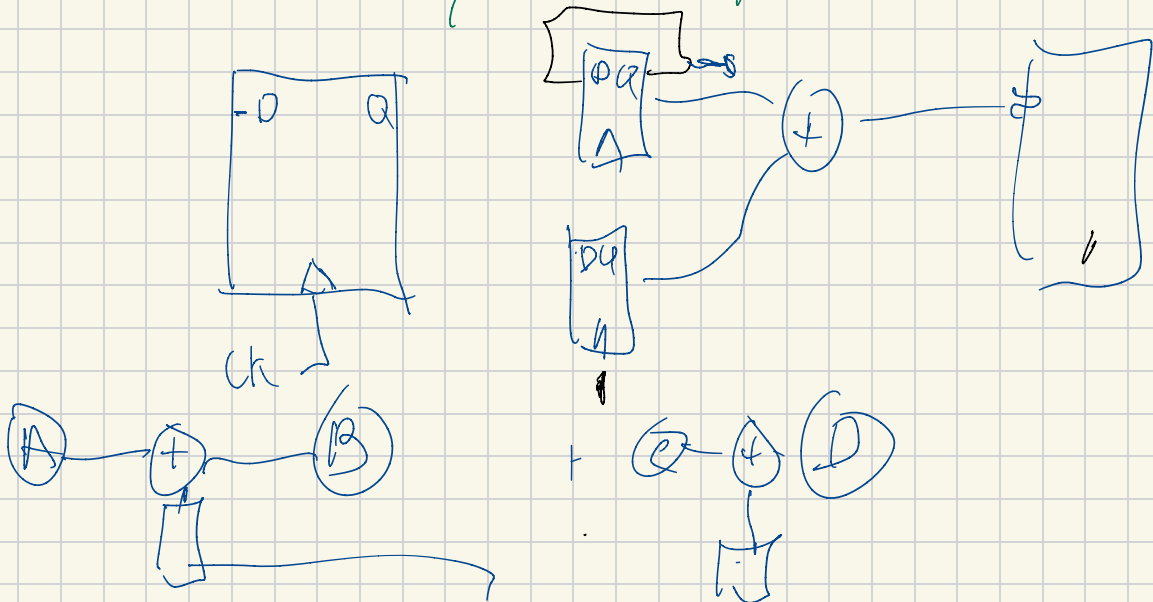
donde:

$$\begin{aligned} T_{\text{data arrival}} &= T_{\text{clk} \rightarrow \text{Q}} + T_{\text{comb}} && \text{(cuándo llega el dato)} \\ T_{\text{data required}} &= T_{\text{period}} - T_{\text{setup}} && \text{(cuándo tiene que llegar)} \end{aligned}$$



$t = t_{\text{setup}}$ - El tiempo que la señal tiene que estar estable antes del flanco positivo de clock

$t = t_{\text{hold}}$ - El tiempo que la señal debe permanecer estable después del flanco.



1.4 Setup check vs Hold check

	Setup Check	Hold Check
Pregunta	¿El dato llega a tiempo?	¿El dato se mantiene estable?
Cuándo falla	Lógica muy lenta	Lógica muy rápida
Peor caso	Corner slow : baja tensión, alta temperatura	Corner fast : alta tensión, baja temperatura
Cómo fixear	Bajar frecuencia, pipeline, optimizar lógica, usar celdas LVT	Agregar buffers de delay
Depende del clock	Sí (más frecuencia = más difícil)	No (es independiente del período)

Hold violations son más peligrosas

Una setup violation se arregla bajando la frecuencia del clock. Una hold violation **no se puede arreglar bajando la frecuencia** — si hay hold violations en el silicio fabricado, el chip no funciona a ninguna frecuencia. Por eso las hold violations se fixean siempre en el flujo de P&R agregando buffers.

1.5 Corners: el chip en distintas condiciones

Un chip en el mundo real funciona a distintas temperaturas y voltajes. Cada combinación extrema es un “corner”:

El sintetizador analiza **ambos corners simultáneamente** (MCM: Multi-Corner Multi-Mode). El escenario `func_slow` verifica setup, el escenario `func_fast` verifica hold.

1.6 SDC: las constraints de timing

El archivo SDC le dice al sintetizador qué timing tiene que cumplir:

```
set PERIOD 1.0
set CONSTRAINT [expr {$PERIOD * 0.10}]

create_clock -period $PERIOD -name clk [get_ports i_clk]
set_input_delay -clock clk $CONSTRAINT [remove_from_collection \
  [all_inputs] [get_ports i_clk]]
set_output_delay -clock clk $CONSTRAINT [all_outputs]
```

- `create_clock`: define un clock con período `PERIOD` en el port `i_clk`.
- `set_input_delay`: modela cuánto tarda el mundo exterior en entregar datos. Le “quita” tiempo al diseño. Con un constraint de 10%, a 1ns de período el input delay es 0.1ns.
- `set_output_delay`: modela cuánto tiempo necesita el mundo exterior para capturar la salida. También le quita tiempo.

2 Tipos de celdas estándar: Multi-Vt

2.1 ¿Qué es el Voltage Threshold (V_t)?

En un transistor MOSFET, el **Voltage Threshold** (V_{th}) es el voltaje mínimo de gate que necesita el transistor para conducir. Es un parámetro fundamental que define el trade-off entre velocidad y consumo de una celda estándar.

Cuanto más bajo el V_{th} , el transistor empieza a conducir antes $\rightarrow$ la celda es más rápida pero tiene más corriente de fuga (*leakage*) en reposo.

2.2 Familia de celdas Multi-Vt

Las foundries modernas ofrecen varias familias de celdas con distintos V_{th} . La disponibilidad varía según tecnología, pero el patrón es siempre el mismo:

2.3 Descripción de cada familia

HVT — High Voltage Threshold Celdas con el umbral de conducción más alto. Son las más lentas pero las que menos corriente de fuga consumen en reposo. El sintetizador las usa en caminos con *slack* holgado para minimizar el consumo estático. En SAED 32nm son el punto de partida de la síntesis.

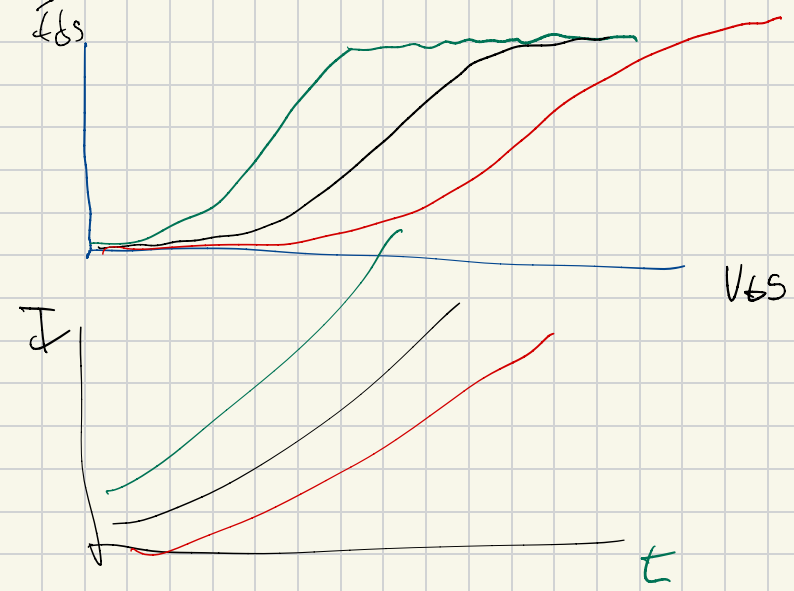
SVT / RVT — Standard / Regular Voltage Threshold El punto intermedio de la familia. Son el baseline de muchas tecnologías (TSMC, GF). Ofrecen buen balance entre performance y potencia. No todas las librerías tienen esta categoría explícitamente; en SAED 32nm la distinción es directamente HVT/LVT.

LVT — Low Voltage Threshold Celdas más rápidas que HVT a expensas de mayor leakage. El sintetizador las introduce cuando el timing se vuelve apretado y el swap HVT $\rightarrow$ LVT es suficiente para cerrar. En el experimento del barrido se observa claramente: a 1 GHz, el 76–93% de las celdas ya son LVT.

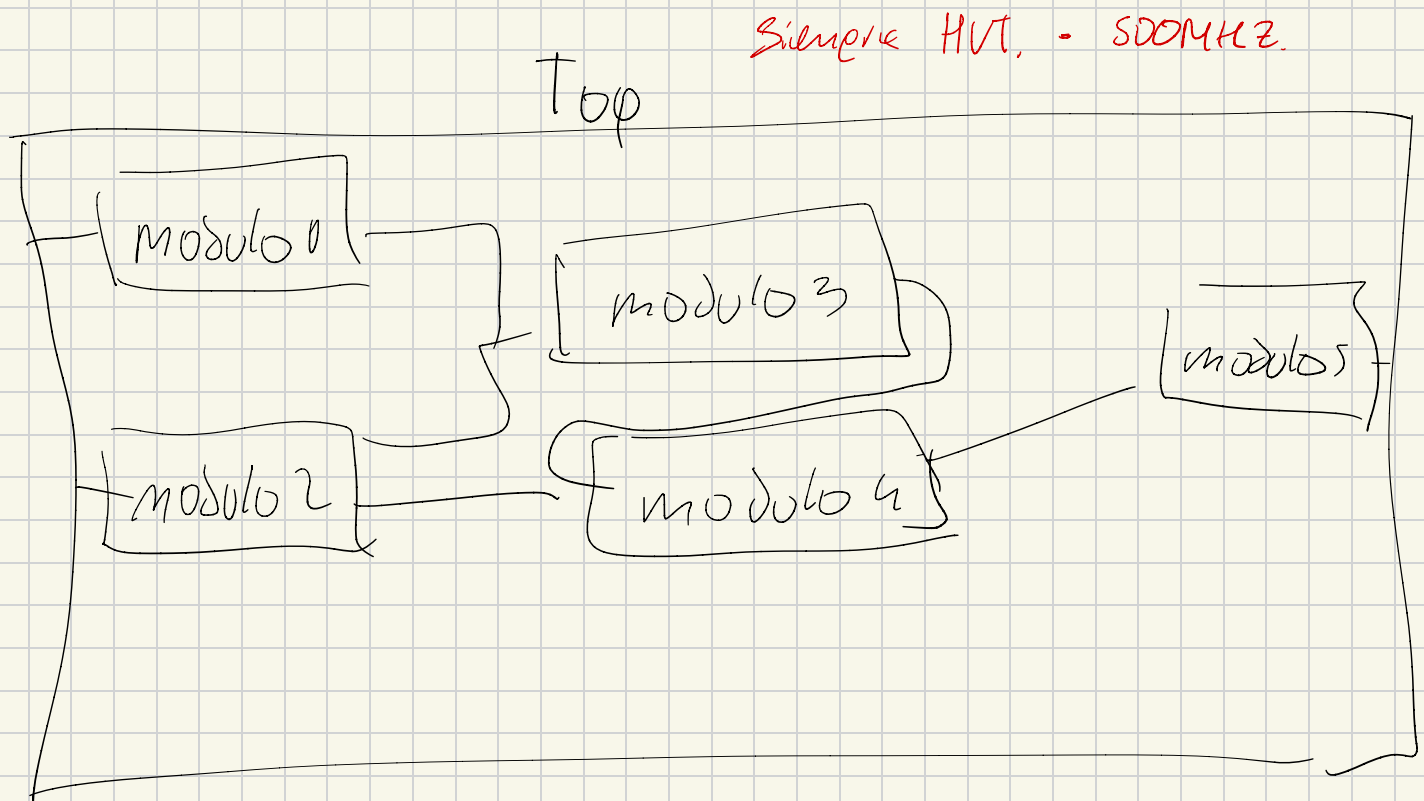
ULVT — Ultra Low Voltage Threshold Disponibles en nodos avanzados (28nm, 16nm, 7nm, etc.). Son significativamente más rápidas que LVT pero con leakage 2–5 $\times$ mayor. Se reservan para los paths críticos más exigentes. No están presentes en SAED 32nm.

ELVT — Extra Low Voltage Threshold Familia extrema usada en nodos sub-20nm (TSMC 7nm/5nm, Samsung 3nm). Ofrecen la máxima velocidad dentro del dominio de voltaje nominal. El leakage puede ser un orden de magnitud mayor que HVT. Solo se justifican en el path más crítico del diseño.

ULV — Ultra Low Voltage **Importante:** ULV no significa “ultra low V_{th} ” sino “ultra low *supply* voltage”. Son celdas diseñadas para operar a V_{DD} muy bajo (sub-threshold: 0.2–0.4V), donde el transistor trabaja por difusión, no por deriva. Se usan en aplicaciones IoT y wearables donde el consumo es crítico. La velocidad cae drásticamente (puede ser 100 $\times$ más lenta que nominal) pero el consumo total (dinámico + estático) se reduce en órdenes de magnitud.



LUT mus repick. + waspoten.
SVT
FET



Simple HVT, - 500MHz.

- Timing Allu o gam no liera timing.
- Inulyo SVT y/o LUT.

2.4 Resumen comparativo completo

Familia	V_{th}	Velocidad	Leakage	V_{DD}	Uso típico
HVT	Alto	*	*	nominal	Paths no críticos, ahorro de potencia
SVT/RVT	Medio	**	**	nominal	Balance general
LVT	Bajo	***	***	nominal	Paths críticos en síntesis
ULVT	Muy bajo	****	****	nominal	Paths ultra-críticos (nodos $\leq 28\text{nm}$)
ELVT	Mínimo	*****	*****	nominal	Path más crítico (nodos $\leq 7\text{nm}$)
ULV	Variable	* (muy baja)	* (muy bajo)	sub-threshold	IoT, wearables, energy harvesting

2.5 Multi-Vt en SAED 32nm vs tecnologías avanzadas

Tecnología	Familias disponibles	Notas
SAED 32nm (académica)	HVT, LVT	Solo dos, ideal para aprender el concepto
TSMC 28nm	HVT, RVT, LVT	Primera generación multi-Vt completa
TSMC 16nm FF+	HVT, RVT, LVT, ULVT	FinFET: V_{th} controlado por geometría
TSMC 7nm	HVT, RVT, LVT, ULVT, ELVT	5 familias, P&R muy complejo
TSMC 3nm GAA	HVT, SVT, LVT, ULVT, ELVT	Gate-All-Around, 5+ familias

Multi-Vt en FinFETs y GAA

En tecnologías planares (como SAED 32nm), el V_{th} se controla mediante dopaje del canal durante la fabricación. En **FinFETs** (16nm–5nm) y **Gate-All-Around** (3nm en adelante), el V_{th} también se controla por la geometría del fin o del nanosheet (altura, ancho), lo que permite un espacio de diseño mucho más continuo. Por eso los nodos avanzados tienen más familias Vt disponibles.

2.6 Cómo el sintetizador usa Multi-Vt

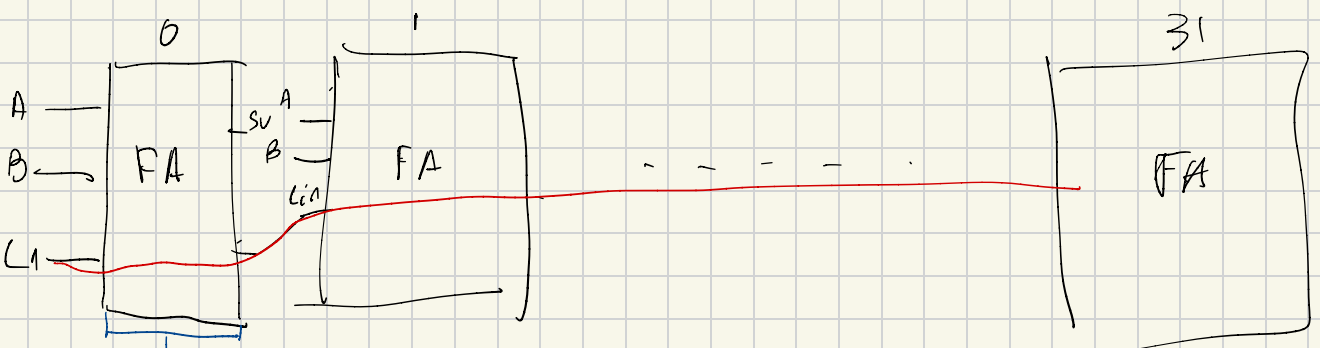
Regla práctica

En un diseño bien optimizado se apunta a usar **70–80% HVT** para mantener bajo el leakage, con LVT solo en los paths críticos. Cuando el porcentaje de LVT supera el 90%, es señal de que la frecuencia objetivo es muy agresiva para la tecnología o el diseño necesita pipeline.

3 Arquitecturas de sumadores

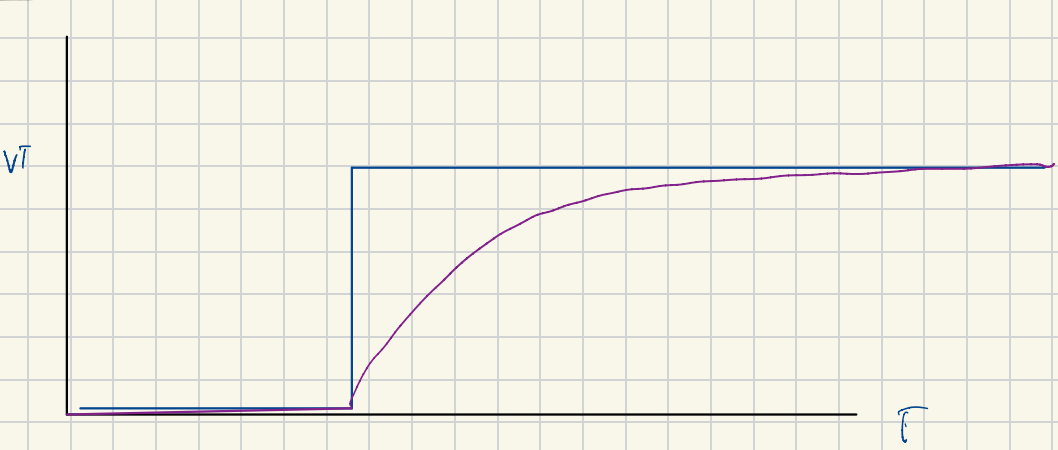
3.1 Ripple Carry Adder (RCA)

Es el sumador más simple: encadena full adders, pasando el carry de uno al siguiente.

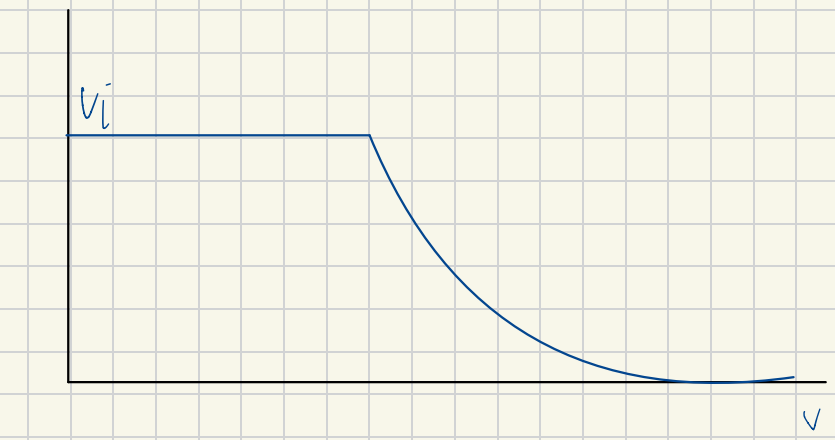
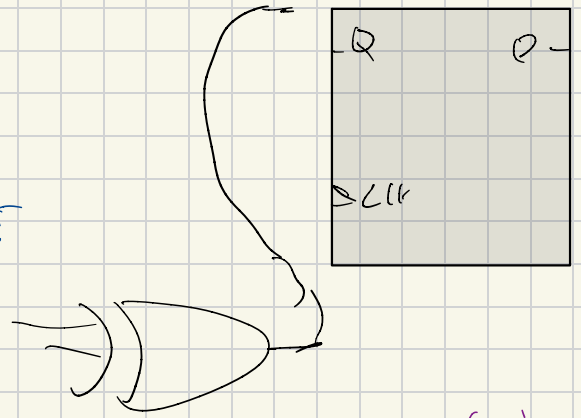


Tiempo de propagación

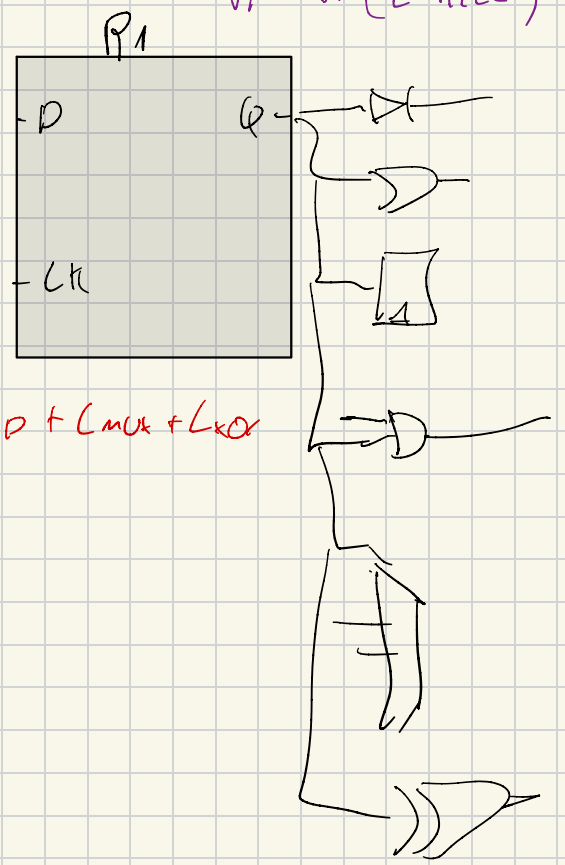
suma total de $T_p = 3,2 \text{ ns}$ mismo



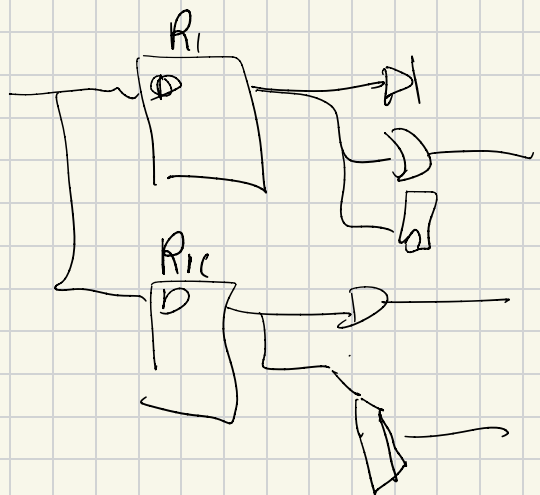
$$V_f = V_i \left(1 - e^{-\frac{t}{R_1 C_1}}\right)$$



$$V_f = V_i \left(e^{-\frac{t}{R_1 C_2}}\right)$$

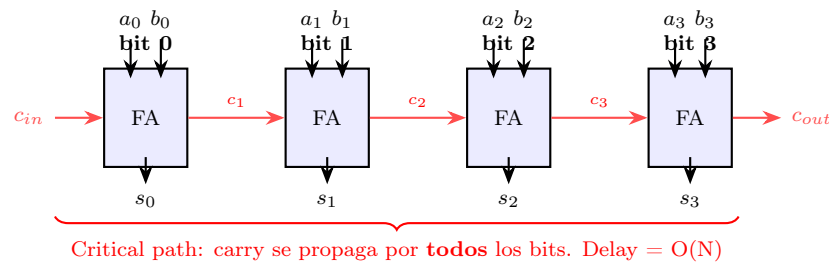


$$C_2 = C_{not} + C_{or} + C_p + C_{and} + C_{mux} + C_{xor}$$





- Replicar el registro 2 veces - [Si la respuesta anterior no tiene prob
do timing?
- Se colocan buffers a la salida.
- 0,4 V - 0-6. Ejemplo 0,4



Ventaja: mínima área (solo N full adders). **Desventaja:** el carry del bit 0 tiene que propagarse por los N full adders hasta llegar al bit $N-1$. Para N grande, esto es muy lento.

Para un sumador de 128 bits, el carry pasa por 128 FAs — inviable a frecuencias altas.

3.2 Carry Lookahead Adder (CLA)

En vez de esperar a que el carry se propague, el CLA **precalcula** los carries en paralelo usando dos señales:

- **Generate** ($G_i = a_i \cdot b_i$): el bit i genera carry independientemente del carry de entrada.
- **Propagate** ($P_i = a_i \oplus b_i$): el bit i propaga el carry de entrada a la salida.

Con estas señales, el carry de cualquier bit se puede calcular directamente:

$$c_1 = G_0 + P_0 \cdot c_0$$

$$c_2 = G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot c_0$$

$$c_3 = G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot c_0$$

Cada carry se calcula en **2 niveles de lógica** (un AND-OR), sin esperar al carry anterior. El delay total es $O(\log N)$ en vez de $O(N)$.

Las variantes más conocidas son:

- **Kogge-Stone:** mínimo delay, máxima área (muchos cables)
- **Brent-Kung:** balance entre delay y área
- **Sklansky:** menos cables pero fan-out alto

3.3 Carry Save Adder (CSA)

El CSA es diferente: no produce un resultado final, sino **dos vectores** (sum y carry) que juntos representan el resultado. No propaga carry en ningún momento.

Se usa cuando necesitas sumar **3 o más operandos** (como en multiplicadores). Cada nivel de CSA reduce 3 operandos a 2, sin propagación de carry. Al final, un CLA resuelve los dos vectores en uno.

Lo que hace Fusion Compiler

Cuando escribís `assign sum = a + b`, el sintetizador elige la arquitectura automáticamente según el timing constraint:

- Constraint relajado → RCA (mínima área)
- Constraint medio → CLA tipo Brent-Kung (balance)
- Constraint apretado → CLA tipo Kogge-Stone (máxima velocidad, más área)

Esto lo hace a través de la librería DesignWare que viene integrada.

4 Experimento: barrido de síntesis

4.1 Setup del experimento

Sintetizamos un sumador parametrizable con registros de entrada variando el **ancho de palabra** y el **período del clock**:

```
module sumador #(parameter int WIDTH = 8) (  
    input  logic          i_clk,  
    input  logic          i_rst,  
    input  logic          i_enable,  
    input  logic [WIDTH-1:0] i_a,  
    input  logic [WIDTH-1:0] i_b,  
    output logic [WIDTH:0]  o_suma  
);  
    logic [WIDTH-1:0] r_a, r_b;  
    always_ff @(posedge i_clk) begin  
        if (i_rst) begin  
            r_a <= '0;  
            r_b <= '0;  
        end else if (i_enable) begin  
            r_a <= i_a;  
            r_b <= i_b;  
        end  
    end  
    assign o_suma = r_a + r_b;  
endmodule
```

Notar que las entradas están registradas (`r_a`, `r_b`) pero la salida es combinacional (`assign`). El critical path es `reg → sumador → output port`.

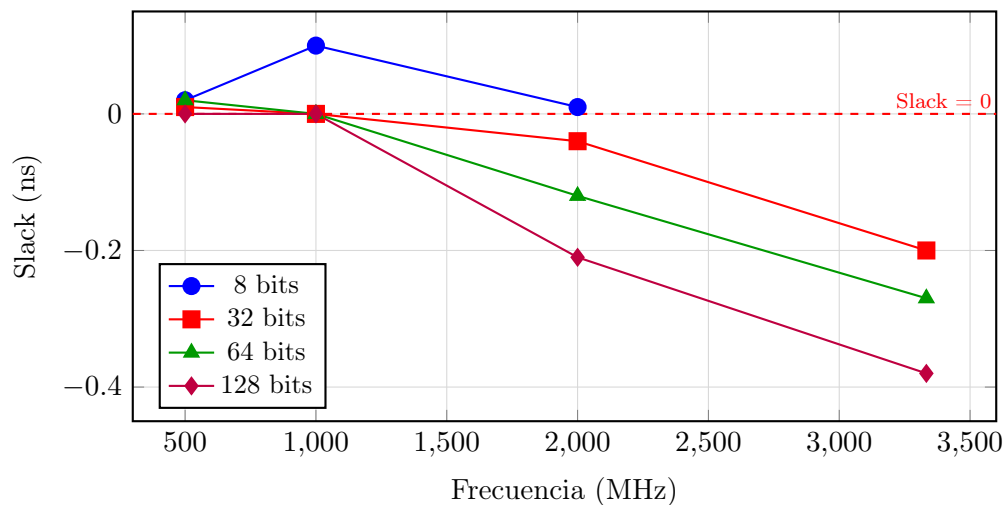
Parámetros del barrido:

- **Anchos:** 8, 32, 64, 128 bits
- **Períodos:** 2.0ns (500MHz), 1.0ns (1GHz), 0.5ns (2GHz), 0.3ns (3.3GHz)
- **Tecnología:** SAED 32nm, Multi-Vt (HVT + LVT)
- **Corners:** Slow (setup), Fast (hold), MCMM

4.2 Resultados

WIDTH	Período	Frecuencia	Slack (ns)	Status	Área	HVT / LVT
8	2.0ns	500 MHz	+0.020	MET	162.9	81% / 19%
8	1.0ns	1 GHz	+0.100	MET	186.3	24% / 76%
8	0.5ns	2 GHz	+0.010	MET	207.9	4% / 96%
8	0.3ns	3.3 GHz	—	TIMEOUT	—	—
32	2.0ns	500 MHz	+0.010	MET	792.7	55% / 45%
32	1.0ns	1 GHz	+0.000	MET	1033.9	13% / 87%
32	0.5ns	2 GHz	-0.040	VIOLATED	1250.4	7% / 93%
32	0.3ns	3.3 GHz	-0.200	VIOLATED	1341.9	2% / 98%
64	2.0ns	500 MHz	+0.020	MET	1712.9	53% / 47%
64	1.0ns	1 GHz	+0.000	MET	1816.6	12% / 88%
64	0.5ns	2 GHz	-0.120	VIOLATED	2767.4	4% / 96%
64	0.3ns	3.3 GHz	-0.270	VIOLATED	2919.1	1% / 99%
128	2.0ns	500 MHz	+0.000	MET	3521.9	53% / 47%
128	1.0ns	1 GHz	+0.000	MET	4570.0	7% / 93%
128	0.5ns	2 GHz	-0.210	VIOLATED	5912.4	7% / 93%
128	0.3ns	3.3 GHz	-0.380	VIOLATED	5536.3	1% / 99%

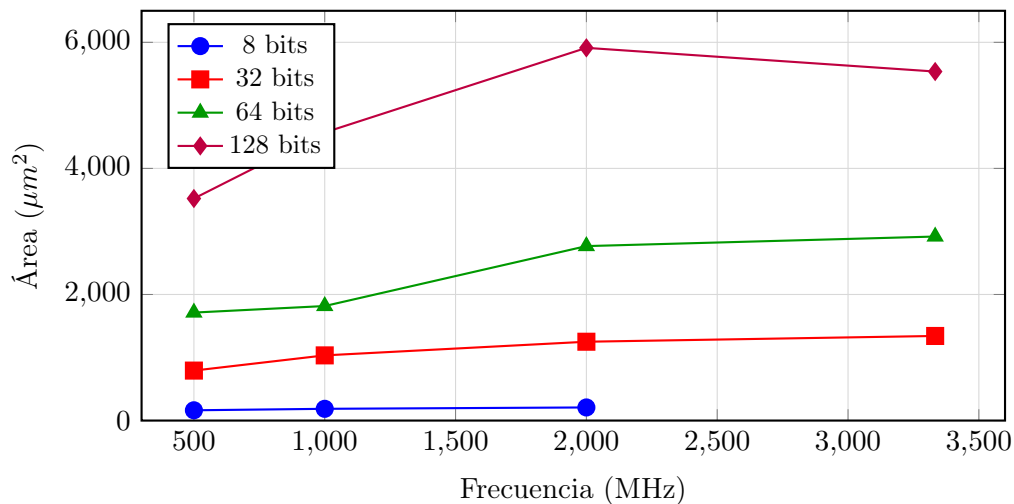
4.3 Análisis: Slack vs Frecuencia



Observaciones:

1. **A 500 MHz todos cierran** — incluso 128 bits. El período de 2ns da margen suficiente para cualquier ancho.
2. **A 1 GHz todos cierran, pero al límite** — los anchos de 32, 64 y 128 bits tienen slack de 0.000ns. Un cable más largo en el layout y rompe.
3. **A 2 GHz solo 8 bits cierra** — para 32+ bits, el carry chain es demasiado largo. La violación empeora con el ancho: -0.04ns (32 bits) vs -0.21ns (128 bits).
4. **A 3.3 GHz nada cierra** — ni siquiera 8 bits (timeout, probablemente no converge).

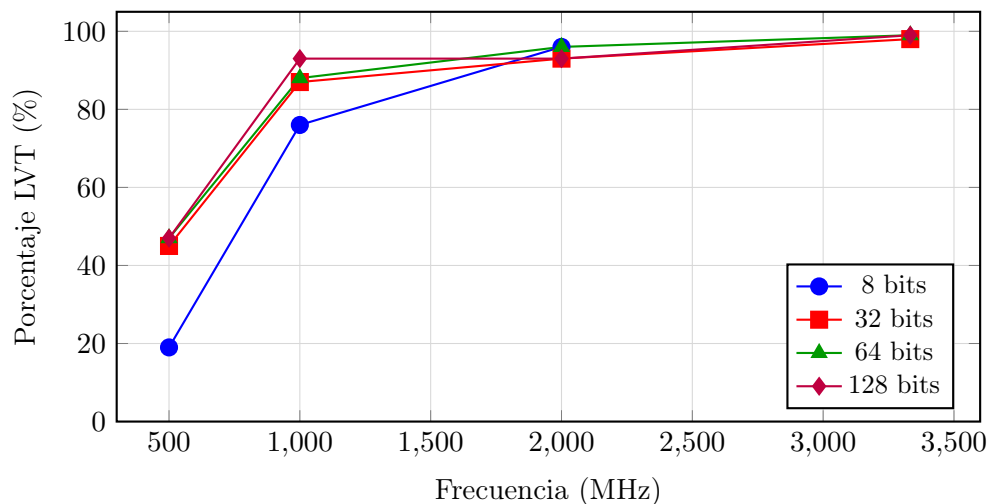
4.4 Análisis: Área vs Frecuencia



Observaciones:

1. **El área crece cuando se aprieta el timing.** Para 128 bits: de $3521 \mu m^2$ (500MHz) a $5912 \mu m^2$ (2GHz) — un aumento del 68%. El sintetizador agrega lógica de carry lookahead que ocupa más espacio.
2. **El aumento no es lineal.** De 500MHz a 1GHz el área crece poco (el sintetizador solo cambia HVT por LVT). De 1GHz a 2GHz crece mucho (cambia la *arquitectura* del sumador).

4.5 Análisis: HVT vs LVT



Observaciones:

1. **A 500 MHz, 8 bits usa 81% HVT.** El timing es holgado — el sintetizador prioriza bajo consumo con celdas lentas.
2. **A 1 GHz, todo pasa a 76–93% LVT.** El sintetizador sacrifica potencia por velocidad.
3. **A 2+ GHz, 96–99% LVT.** Ya no quedan celdas HVT que cambiar. La única forma de ir más rápido es cambiar la arquitectura del sumador (CLA en vez de RCA). Cuando ni eso alcanza, el timing viola.

La historia que cuentan los datos

El sintetizador tiene **tres herramientas** para cerrar timing, y las usa en este orden:

- 1. Swap HVT → LVT** (barato): cambiar celdas lentas por rápidas. No cambia el área, pero aumenta el consumo. Es lo primero que hace.
- 2. Cambiar la arquitectura** (caro): reemplazar el sumador RCA por un CLA. Aumenta el área significativamente, pero reduce el delay de $O(N)$ a $O(\log N)$.
- 3. Rendirse** (violated): cuando ni las celdas más rápidas ni la mejor arquitectura alcanzan, el timing viola. La única salida es bajar la frecuencia o hacer pipeline.

4.6 Frecuencia máxima por ancho

WIDTH	Freq. máxima que cierra	Observación
8 bits	≥ 2 GHz	Cierra cómodo incluso a 2GHz
32 bits	~ 1 GHz	Cierra justo a 1GHz (slack = 0.000)
64 bits	~ 1 GHz	Cierra justo a 1GHz (slack = 0.000)
128 bits	~ 1 GHz	Cierra justo a 1GHz (slack = 0.000)

Es interesante que 32, 64 y 128 bits los tres cierran a 1GHz. Esto sugiere que el sintetizador está usando un tree adder (CLA) que tiene delay $O(\log N)$ — la diferencia entre $\log_2(32) = 5$ y $\log_2(128) = 7$ niveles es chica comparada con el delay total del path.

5 Leyendo un timing report

Ejemplo real del barrido (8 bits, 0.5ns):

Startpoint: r_b_reg[5] (flip-flop clocked by clk)
 Endpoint: o_suma[7] (output port clocked by clk)
 Path Type: max (setup check)

Point	Incr	Path

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	0.00	0.00
r_b_reg[5]/CLK (DFFSSRX1_LVT)	0.00	0.00 r
r_b_reg[5]/Q (DFFSSRX1_LVT)	0.10	0.10 f <- Tclk->Q
A250/Y (OR2X1_LVT)	0.05	0.15 f <- carry logic
ctmTdsLR_1_830/Y (OA21X1_LVT)	0.05	0.20 f <- lookahead
ctmTdsLR_1_832/Y (AND2X1_LVT)	0.04	0.24 f <- lookahead
ctmTdsLR_3_834/Y (AO221X1_LVT)	0.06	0.30 f <- lookahead
A153/CO (FADDX1_LVT)	0.06	0.35 f <- full adder
A154/S (FADDX1_LVT)	0.09	0.44 r <- full adder
o_suma[7] (out)	0.00	0.44 r
data arrival time		0.44
data required time		0.45

slack (MET)		0.01

Qué nos dice:

1. El path es **reg2out**: desde un registro (`r_b_reg[5]`) hasta un output port (`o_suma[7]`).
2. Todas las celdas son **_LVT** — el sintetizador puso 96% LVT porque el timing es apretado.
3. La arquitectura es **híbrida**: celdas compound (**0A21**, **A0221**) para carry lookahead en los bits bajos, y full adders (**FADDX1**) para los bits finales.
4. $T_{\text{clk} \rightarrow \text{Q}} = 0.10\text{ns}$ (el flip-flop). La lógica combinacional total = 0.34ns . Slack = $+0.01\text{ns}$ — cierra por un pelo.

6 Conclusiones

1. **El timing determina todo.** El mismo RTL ($a + b$) produce hardware completamente diferente según el constraint de frecuencia.
2. **Hay tres niveles de optimización:** swap Vt (barato), cambiar arquitectura (caro en área), rendirse (violated).
3. **Multi-Vt es una herramienta de balance:** HVT para low-power en paths relajados, LVT/ULVT/ELVT para máxima velocidad en paths críticos. En tecnologías avanzadas, el espacio de diseño Multi-Vt es mucho más rico.
4. **El ancho de palabra importa**, pero menos de lo esperado gracias al carry lookahead. La diferencia entre 32 y 128 bits es mucho menor que $4\times$ porque el delay escala como $O(\log N)$, no como $O(N)$.
5. **Área y velocidad son un trade-off:** ganar velocidad siempre cuesta área (y potencia). El diseñador tiene que elegir el punto de operación óptimo.
6. **Las hold violations no dependen de la frecuencia.** Son un problema de la implementación física que se resuelve en P&R, no en síntesis.